

Moderated Video Q&A Platform

Build roadmap: mentor video learning + AI Q&A, gated by a toxicity moderation model.

1. Product Overview

Mentors upload videos. Students watch them and can ask an AI questions about the content (summaries, explanations, overviews) similar to NotebookLM. Every question a student submits is first checked by a toxicity moderation model — the AI never answers a toxic or abusive question, and the same check protects comments and chat elsewhere on the platform.

Two AI systems, one gate between them:

- **Q&A / summarization system** — a retrieval-augmented generation (RAG) pipeline over mentor video transcripts.
- **Moderation system** — a text classifier trained on the labeled toxicity dataset, sitting in front of every user-submitted question or comment.

Request flow:

Student asks a question
→ Moderation model checks the text
→ Toxic? Block. Do not call the AI. Show a policy message.
→ Clean? Retrieve relevant video transcript chunks
→ Send question + context to the LLM
→ Return the answer to the student

2. Build Roadmap

Phase 0 — Data foundation (done)

- Sourced the Azerbaijani toxicity dataset (~75k labeled comments, 7 categories).
- Cleaned label conflicts on near-duplicate text (77 groups fixed).
- Split into train/val/test (80/10/10), stratified and duplicate-safe.

Phase 1 — Moderation MVP

- Trained a TF-IDF + Logistic Regression baseline classifier (~79% toxicity accuracy).
- Wrapped it as a `moderate(text)` function returning toxic/not + flagged categories.
- Demonstrated the gate: `ask_question()` calls `moderate()` before any AI call.

Phase 2 — Video ingestion pipeline

- Transcribe mentor videos (e.g. Whisper) into text.
- Chunk transcripts into passages sized for retrieval.
- Generate embeddings for each chunk and store in a vector database (e.g. pgvector, Pinecone, Weaviate).

Phase 3 — RAG Q&A system

- On a student question, embed it and retrieve the most relevant transcript chunks.
- Send question + retrieved context to an LLM (e.g. Claude) to generate the answer.
- Support both direct Q&A and "summarize this video" style requests.

Phase 4 — Unify moderation + Q&A

- Every question passes through the moderation gate before reaching the RAG pipeline.
- Apply the same gate to comments and any other free-text surfaces on the platform.
- Log blocked attempts for review, without storing them as normal content.

Phase 5 — Strengthen & scale

- Fine-tune a transformer model (e.g. XLM-R) on a GPU for higher accuracy, especially on rare categories like threats and severe toxicity.
- Collect real moderator-reviewed examples over time and periodically retrain.
- Add per-category thresholds: auto-block high-confidence cases, route borderline cases to human review.
- Monitor false-positive rate across topics/groups to catch model bias.

3. Suggested Tech Stack

Layer	Options
Video transcription	Whisper (open-source or API)
Embeddings + vector store	OpenAI/Voyage embeddings + pgvector, Pinecone, or Weaviate
LLM for Q&A / summaries	Claude API (Sonnet for quality, Haiku for cost/latency)
Moderation model (MVP)	TF-IDF + Logistic Regression (already built)
Moderation model (scaled)	Fine-tuned XLM-R or similar multilingual transformer
Backend service	FastAPI (or existing platform backend) exposing /moderate and /ask

4. Key Risks & Considerations

- **Rare-category precision:** the baseline model is weak on threats and severe toxicity (few real examples). Don't auto-block on those alone yet — route to human review.
- **Evasion:** toxic text can be obfuscated (spacing, symbols, typos). Test the moderation model against obfuscated variants before launch.
- **Bias:** monitor false-positive rates across topics/groups — moderation models can over-flag reclaimed language or legitimate political/religious speech.
- **Don't over-trust synthetic augmentation:** it gave only marginal gains in testing. Real labeled data for rare categories is more valuable.
- **Compute:** the transformer fine-tune (Phase 5) needs a GPU — the baseline runs fine on CPU and is enough to ship an MVP.

5. Immediate Next Steps

- Wrap the existing baseline model as a small API (/moderate endpoint).
- Pick a vector DB and transcription tool, and ingest a handful of pilot videos.
- Build the ask_question() gate against real LLM calls (swap the stubbed prompt in demo_moderated_qa_pipeline.py for a live Claude API call).
- Once GPU access is available, fine-tune the transformer model and A/B it against the baseline before switching over.

Generated as a working overview — treat phase boundaries as flexible, not fixed milestones.